

Inferência de rede neural densa (MLP forward pass)

Kaik Henrique N. Soares¹, Paulo Henrique C. Silva²

¹Bacharelado em Ciências da computação – Instituto Federal Goiano (IFGO)
CEP 76200-000 – Iporá – GO – Brasil

²Bacharelado em Ciências da computação – Instituto Federal Goiano (IFGO)
CEP 76200-000 – Iporá – GO – Brasil

{kaik.henrique@estudante.ifgoiano.edu.br, paullo.castro@estudante.ifgoiano.edu.br}

Abstract. *This work aims to implement the forward pass of a Multi-Layer Perceptron (MLP) neural network using three distinct approaches: CPU processing, custom CUDA implementation, and GPU implementation using the PyTorch library. The neural network architecture consists of an input layer with 784 neurons, two hidden layers with 128 and 64 neurons, and an output layer with 10 neurons. Tests were performed using different batch sizes (1, 64, 256, and 1024) to analyze the computational performance of the implemented approaches. The results demonstrated that the GPU achieved significant performance gains in scenarios with higher parallelism, while PyTorch obtained the best results due to the internal optimizations provided by the cuBLAS library.*

Resumo. *Este trabalho tem como objetivo implementar o forward pass de uma rede neural do tipo Multi-Layer Perceptron (MLP) utilizando três abordagens distintas: processamento em CPU, implementação CUDA customizada e implementação utilizando a biblioteca PyTorch em GPU. A arquitetura da rede neural foi composta por uma camada de entrada com 784 neurônios, duas camadas ocultas com 128 e 64 neurônios e uma camada de saída com 10 neurônios. Foram realizados testes utilizando diferentes tamanhos de batch size, sendo eles (1,64,256,1024), para análise de desempenho computacional entre as abordagens implementadas. Os resultados demonstraram que a GPU apresentou ganhos significativos de desempenho em cenários com maior paralelismo, enquanto o PyTorch obteve os melhores resultados devido às otimizações internas realizadas pela biblioteca cuBLAS.*

1. Introdução

O objetivo deste trabalho é demonstrar como o avanço arquitetural das Unidades de Processamento Gráfico (GPUs) e a evolução das linguagens e frameworks de alto nível atuam como pilares no desenvolvimento de novas tecnologias baseadas em Inteligência Artificial [Owens et al. 2008, LeCun et al. 2015]. Para tanto, analisa-se o desempenho do processo de inferência (forward pass) de um Perceptron Multicamadas (MLP) [Haykin 2009] sob três abordagens distintas: execução sequencial em CPU, execução em GPU por meio de um kernel customizado em CUDA escrito do zero, e execução otimizada em GPU utilizando o framework PyTorch. Os resultados experimentais de tempo de execução são contrastados e discutidos, correlacionando a eficiência computacional de cada ambiente à sua respectiva intensidade aritmética e capacidade de paralelismo massivo.

2. Fundamentação Teórica

A algebra de um Perceptron Multicamadas (MLP) é baseada em:

$$Y = \text{ReLU}(X \cdot W + b)$$

Onde o X é a matriz de dados de entrada(batch size), W representa a matriz de pesos sinápticos, b corresponde ao vetor de vieses e o ReLU é a função de ativação Unidade Linear Retificada. A intensidade aritmetica pode ser definida como a razão entre operações de pontos flutuantes (FLOPs) realizadas e o total de bytes transferidos da memória principal (Bytes). Essa relação é governada pelo modelo roofline, [Williams et al. 2009] um modelo visual de desempenho responsável por informar os limites teóricos de vazão que um determinado programa consegue pode rodar em um determinado arquitetura de hardware [Kirk and Hwu 2016]. Esse modelo é responsável por dividir os gargalos em duas regiões fundamentais:

1- Memory-bound: Onde o hardware gasta mais tempo esperando dados trafegarem pelo barramento do que processando. Ocorre em batches pequenos.

2- Compute-Bound: Onde os núcleos de processamento atingem o pico de uso porque os dados já estão nas caches internas e são reutilizados várias vezes. Ocorre em batches grandes.

3. Trabalhos Relacionados

A otimização e a análise de desempenho de algoritmos de Deep Learning em diferentes arquiteturas de hardware têm sido amplamente discutidas na literatura recente. O impacto do tamanho do lote (*batch size*) no rendimento computacional foi investigado por [Goyal et al. 2017], que demonstraram que, embora lotes grandes otimizem a taxa de processamento (*throughput*) e a reutilização de dados na GPU, eles impõem desafios matemáticos quanto à convergência e à precisão do modelo durante o treinamento.

No que tange à comparação direta entre ecossistemas de hardware, [Shi et al. 2016] realizaram um extenso *benchmarking* avaliando o desempenho de inferência e treinamento em CPU e GPU utilizando frameworks populares, como Caffe, TensorFlow e o próprio PyTorch. Os autores evidenciaram que bibliotecas de álgebra linear de alto nível (como a cuBLAS) conseguem explorar o paralelismo massivo de GPUs de forma muito superior a códigos ingênuos, corroborando a importância de otimizações de baixo nível na memória do dispositivo.

Por fim, o Modelo Roofline tem sido frequentemente adotado para mapear os limites de desempenho de Redes Neurais Convolucionais (CNNs) e MLPs. Em especial, [Yang et al. 2017] aplicaram a metodologia Roofline para diagnosticar gargalos em aceleradores de hardware, demonstrando quantitativamente que camadas densas (*fully connected*), como as da MLP avaliada neste trabalho, transitam rapidamente de um regime estritamente *memory-bound* para *compute-bound* à medida que a densidade do lote computado é escalada.

4. Metodologia

Para avaliar o impacto do tamanho do lote (*batch size*) no desempenho computacional, o experimento consistiu na execução da etapa de inferência (*forward pass*) de uma rede

neural artificial do tipo Perceptron Multicamadas (MLP). O modelo foi estruturado para simular a classificação de imagens com dimensões equivalentes às do conjunto de dados MNIST (28×28 pixels).

4.1. Arquitetura da MLP

A topologia da rede avaliada foi padronizada com três camadas densas sequenciais (totalmente conectadas), descritas a seguir:

- **Camada de Entrada:** Composta por 784 neurônios, correspondentes às imagens vetorizadas de tamanho 28×28 .
- **Camada Oculta 1:** Contendo 128 neurônios, utilizando a função de ativação Unidade Linear Retificada (ReLU).
- **Camada Oculta 2:** Contendo 64 neurônios, também utilizando ativação ReLU.
- **Camada de Saída:** Contendo 10 neurônios lineares, representando o total de classes do problema de classificação.

4.2. Ambiente de Hardware e Software

A validação experimental foi conduzida utilizando a infraestrutura de computação em nuvem do Google Colab. O ambiente de execução contou com as seguintes especificações físicas e lógicas:

- **Unidade Central de Processamento (CPU):** Processador Intel Xeon operando com frequência base de 2.20 GHz.
- **Unidade de Processamento Gráfico (GPU):** Placa de vídeo NVIDIA Tesla T4, dotada de arquitetura Turing, contendo 2560 núcleos CUDA, 320 Tensor Cores e 16 GB de memória de vídeo dedicada VRAM GDDR6.
- **Ambiente de Software:** Sistema operacional Linux Ubuntu, compilador GCC, NVIDIA CUDA Toolkit versão 12 e linguagem de programação Python integrada ao framework PyTorch.

4.3. Estratégias de Implementação

A inferência da rede foi desenvolvida sob três paradigmas e linguagens distintas:

1. **Código CPU Sequencial:** Desenvolvido em linguagem C++, utilizando loops aninhados estruturados de forma puramente sequencial para a multiplicação de matrizes e aplicação de *bias* e ativação. Serviu como a linha de base (*baseline*) do experimento.
2. **Código GPU CUDA Customizado:** Escrito em C++/CUDA nativo (.cu), implementando um *kernel* do zero. A estratégia adotou o mapeamento de blocos bidimensionais de *threads* (16×16) e aplicou a técnica de fusão de operadores (*Operator Fusion*), realizando o cálculo do produto matricial, a soma do vetor de *bias* e a aplicação da ReLU em um único ciclo de execução do *kernel*, reduzindo acessos à memória global da GPU.
3. **Código PyTorch GPU:** Desenvolvido em Python através da API nativa do PyTorch. Os tensores e pesos foram alocados diretamente no dispositivo CUDA ('device='cuda'), fazendo uso implícito da biblioteca industrial de alto desempenho cuBLAS [?], que realiza otimizações dinâmicas de acesso coalescente de memória e ativação de *Tensor Cores*.

Para fins de mensuração, o tempo de execução foi coletado exclusivamente sobre as etapas de cálculo aritmético da inferência, desconsiderando as etapas iniciais de carregamento do arquivo de pesos para a memória. Foram testados de forma incremental os *batch sizes* de 1, 64, 256 e 1024.

5. Implementação

Para implementar toda a lógica do trabalho, foi separado em três divisões, sendo eles: CPU sequencial, GPU customizado e GPU com pytorch.

5.1. CPU Sequencial

O código é iniciado com a inclusão das bibliotecas `stdio.h`, `stdlib.h` e `chrono`. A biblioteca `chrono` é utilizada para realizar a medição do tempo de execução com alta precisão, permitindo avaliar o desempenho computacional da implementação na CPU. Em seguida, são definidas as dimensões da rede neural, sendo:

1. 784 neurônios na camada de entrada;
2. 128 neurônios na primeira camada oculta;
3. 64 neurônios na segunda camada oculta;
4. 10 neurônios na camada de saída.

Além disso, define-se o tamanho do lote (*batch size*), que representa a quantidade de amostras processadas simultaneamente, podendo assumir valores como 1, 64, 256 e 1024. A função de ativação utilizada nas camadas ocultas é a ReLU (Rectified Linear Unit), responsável por introduzir não linearidade ao modelo. O programa utiliza alocação dinâmica de memória por meio da função `malloc`, reservando espaço para armazenar as matrizes de entrada, pesos, bias e saídas intermediárias da rede neural. A propagação direta (*forward propagation*) é realizada utilizando estruturas de repetição aninhadas (`for`). O primeiro laço percorre as amostras do lote, o segundo percorre os neurônios da camada e o terceiro realiza o produto escalar entre entradas e pesos sinápticos. Após o cálculo das duas camadas ocultas, aplica-se a função de ativação ReLU. Já na camada de saída, é realizada apenas a operação linear, sem aplicação de função de ativação. O maior custo computacional da implementação está associado às operações de multiplicação de matrizes, responsáveis pela maior parte do processamento da rede neural.

5.2. GPU customizado

O código inicia incluindo as bibliotecas `stdio.h`, `stdlib.h` e `cuda_runtime.h`. A biblioteca `cuda_runtime.h` fornece acesso às funcionalidades da API CUDA Runtime, permitindo a utilização da GPU para processamento paralelo.

Define-se a arquitetura da rede neural MLP contendo:

1. 784 neurônios na camada de entrada;
2. 128 neurônios na primeira camada oculta;
3. 64 neurônios na segunda camada oculta;
4. 10 neurônios na camada de saída.

Também é definido o tamanho do lote (batch size), representando a quantidade de amostras processadas simultaneamente pela GPU. O código utiliza mapeamento bidimensional de threads para identificar qual elemento da matriz de saída será calculado por cada thread CUDA, utilizando as seguintes equações:

$$R = (By \cdot Dy + Ty)$$

$$C = (Bx \cdot Dx + Tx)$$

onde:

1. R representa a linha da matriz de saída;
2. C representa a coluna da matriz de saída;
3. B representa o índice do bloco na grade;
4. D representa a dimensão do bloco;
5. T representa o índice da thread dentro do bloco.

Foi implementada também uma função responsável pela inicialização das matrizes com valores aleatórios. A alocação de memória na GPU é realizada utilizando `cudaMalloc`, enquanto a transferência de dados entre CPU e GPU é realizada através da função `cudaMemcpy`, permitindo que os cálculos sejam executados paralelamente na GPU.

5.3. GPU com pytorch

O código inicia-se importando as bibliotecas `torch`, `torch.nn` e `time`, a biblioteca `pytorch` é utilizada para construção e execução da rede neural e a biblioteca `time` é utilizada para a medição do tempo. Define-se a mesma arquitetura MLP utilizada em CUDA contendo:

1. 784 neurônios na camada de entrada;
2. 128 neurônios na primeira camada oculta;
3. 64 neurônios na segunda camada oculta;
4. 10 neurônios na camada de saída;

Também é definido o batch size nos tamanhos: 1, 64, 256, 1024 permitindo comparar a quantidade de amostras simultaneamente. Diferente da GPU customizada, a GPU com `pytorch` utiliza uma biblioteca chamada `cuBLAS` que deixa o procedimento mais eficiente e rápido a rede neural é implementada utilizando três funções e um método específicos, sendo elas:

1. `nn.Sequential`;
2. `nn.Linear`;
3. `nn.ReLU`;
4. `.eval`;

Podendo ser explicadas assim: `nn.Sequential` contém camadas lineares, (`nn.linear`) e funções de ativação ReLU (`nn.ReLU`). O método `.eval` coloca a rede neural em modo de inferência removendo possíveis comportamentos de aprendizagem. Os dados de entrada são gerados aleatoriamente utilizando `torch.randn` afim de simular imagens com lotes de dimensão 784. Antes da medição de tempo é realizado um processo chamado Warm up executando o modelo algumas vezes para estabilizar o funcionamento da GPU afim de evitar inferências causadas pela iniciação dos Kernels e CUDA. A medição de tempo é feito através da função `time.perf_counter`, enquanto o `torch.cuda.synchronize` garante que todas as operações do teste sejam feitas com sucesso.

6. Resultados e Discussão

Após a execução dos testes utilizando CPU, GPU customizada com tecnologia NVIDIA T4 e GPU utilizando PyTorch também executada em uma NVIDIA T4, foi possível obter resultados relevantes em relação ao desempenho computacional da rede neural MLP. A Tabela abaixo apresenta de forma explícita o tempo de execução de cada batch size nas três implementações analisadas.

Batch	CPU (ms)	GPU (ms)	GPU PyTorch (ms)
1.0	0.1528	30.2633	0.1266
64.0	0.1565	0.4441	0.1549
256.0	0.1632	0.4608	0.1427
1024.0	0.1597	0.0313	0.1781

Figura 1. Comparativo de tempo de inferência da MLP entre CPU, CUDA Customizado e PyTorch.

Através da tabela pode-se identificar que os tempos no batch 1 a CPU obteve um tempo menor que a GPU customizada, isso ocorre porque a CPU utiliza o formato sequencial para realizar atividades enquanto a GPU customizada tem todo o trabalho de iniciar os kernels e possui o trabalho para buscar dados na memória VRAM [Hwu et al. 2022]. A implementação utilizando PyTorch apresentou o melhor desempenho entre as abordagens analisadas [Shi et al. 2016]. Isso ocorre devido ao uso da biblioteca cuBLAS [Paszke et al. 2019], que fornece operações altamente otimizadas para multiplicação de matrizes e processamento paralelo na GPU, reduzindo gargalos computacionais e melhorando a eficiência da inferência. No batch 256, nota-se que a GPU customizada tem um tempo de inferência algumas vezes maior que a da CPU, isso ocorre porque a GPU customizada no batch 256 está no pior cenário possível há um engarrafamento na VRAM. As threads aumentaram e todas as threads tentam acessar o barramento ao mesmo tempo [Hwu et al. 2022]. No batch 1024 o cenário é perfeito para a GPU customizada, o seu tempo médio é o menor entre as outras duas implementações e isso ocorre porque ela trabalha em sua capacidade máxima enquanto um grupo de threads espera a memória, a GPU tem milhares de outras prontas para calcular. O hardware finalmente entra no regime Compute-Bound e trabalha na velocidade máxima por imagem. A CPU não consegue acompanhar a exigência de um processamento muito rápido e acaba tendo um tempo de processamento bem demorado.

7. Conclusões e Trabalhos Futuros

Após os testes efetuados pode-se afirmar que foi cumprido com sucesso o tema proposto, através da programação em uma linguagem de baixo nível e de alto nível foi possível verificar que as linguagens atuais utilizam mais formas de agilizar processos de forma simples e prática sem a necessidade de um estudo mais aprofundado para obter um resultado satisfatório. Após esse trabalho concluído que, uma GPU com CUDA customizada demora menos tempo quando trabalhada em sua capacidade máxima diferente de uma CPU que tem o seu desempenho reduzido pelo motivo de não grande capacidade

de processamento, enquanto a GPU com CUDA em pytorch mantém seu desempenho muito superior em todos os batch pelo fato de possuir o cuBLAS que fornece operações altamente otimizadas para multiplicação de matrizes e processamento paralelo na GPU [Shi et al. 2016]. Para uma futura melhoria na GPU customizada poderia-se otimizar o Kernel utilizando memória compartilhada e a técnica de Tiling através dessa otimização seria possível reduzir expressivamente os engarrafamentos na VRAM podendo assim obter tempos melhores, evitando gargalos tão grandes como no batch de 256.

Referências

- Goyal, P., Dollár, P., Girshick, R., Noordhuis, P., Wesolowski, L., Gallo, A., LeCun, Y., and He, K. (2017). Accurate, large minibatch sgd: Training imagenet in 1 hour. *arXiv preprint arXiv:1706.02677*.
- Haykin, S. (2009). *Redes Neurais: Princípios e Prática*. Bookman Editora, 3rd edition.
- Hwu, W.-m. W., Kirk, D. B., and El-Hajj, I. (2022). *Programming Massively Parallel Processors: A Hands-on Approach*. Morgan Kaufmann.
- Kirk, D. B. and Hwu, W.-m. W. (2016). *Programming Massively Parallel Processors: A Hands-on Approach*. Morgan Kaufmann, 3rd edition.
- LeCun, Y., Bengio, Y., and Hinton, G. (2015). Deep learning. *Nature*, 521(7553):436–444.
- Owens, J. D., Houston, M., Luebke, D., Green, S., Stone, J. E., and Phillips, J. C. (2008). Gpu computing. *Proceedings of the IEEE*, 96(5):879–899.
- Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Kopf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., Bai, J., and Chintala, S. (2019). Pytorch: An imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems*, 32.
- Shi, S., Wang, Q., Xu, P., and Chu, X. (2016). Benchmarking state-of-the-art deep learning software tools. In *2016 7th International Conference on Cloud Computing and Big Data (CCBD)*, pages 99–104. IEEE.
- Williams, S., Waterman, A., and Patterson, D. (2009). Roofline: an insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76.
- Yang, C., Chang, C., Wang, Y., Hwu, W.-m., and Chen, D. (2017). Neural network implementation on fpgas: A roofline model approach. In *Proceedings of the 2017 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*, pages 123–128.